



# From Agile to DevOps: Smart Skills and Collaborations

Aymeric Hemon<sup>1,2</sup> · Barbara Lyonnet<sup>2</sup> · Frantz Rowe<sup>2,3</sup> · Brian Fitzgerald<sup>4</sup>

Published online: 7 March 2019

© Springer Science+Business Media, LLC, part of Springer Nature 2019

## Abstract

Although agile software development approaches have become increasingly prevalent, many organizations, have found they were not able to achieve a more frequent release cadence, largely due to different departmental functions operating in silos. In an effort to remove these silos, companies have moved towards DevOps. As digitalization continues, companies increasingly implement DevOps. We suggest three different stages in the agile to DevOps transition: agile, continuous integration, and continuous delivery. Based on an in-depth case study in an organization with several years' experience in DevOps, we identify a fundamental disruption in the soft skills that software teams are expected to possess, and in the patterns of collaboration among teams. Arguably, smartness may be characterized as being flexible, teaming up with people who have a different profile, belonging to a different function, and achieving a quicker delivery schedule. In light of this, we argue that DevOps leads to greater smartness for the Information Systems function.

**Keywords** DevOps · Agile · Smartness · Roles · Skills · Collaboration

## 1 Introduction

There has been considerable interest in the smart concept over the past two decades, to the extent that we have examples across the whole A-Z spectrum, from Smart Aging to Smart Z-wave home monitoring. Alter (2018) provides a good overview of the exhaustive range of smart 'things'. This special issue seeks to adopt a more holistic view, that of 'smart work-

ing, living and organizing' which is focused on the integration of various smart initiatives to fulfill this larger purpose. Our study addresses the smart working and smart organizing aspects of the DevOps approach to software development.

The majority of companies, over 90% according to a recent estimate (Version One 2018) are utilizing an agile approach for software development. Agile methods seek to ensure a close link between the customer and developers to help ensure that software meets market needs. Agile methods also strive for a more rapid release schedule. However, while agile methods can achieve a more frequent cadence of development of software, a bottleneck has emerged in that the Operations function (Ops), who coordinate the actual release of software in organizations, are typically not aligned with the Development function (Dev). As a consequence this can lead to long delays in software releases to customers. In order to solve this problem, Debois advocated a tighter integration between the Dev and Ops functions which is termed DevOps (Brunnert et al. 2015; Debois 2009, 2011).

In the past few years, companies are also increasingly moving towards a DevOps approach. Some authors analyzed barriers in the transition from waterfall to agile, to continuous integration and to continuous deployment (Holmström Olsson et al. 2012). However, there is great variety in the extent to which the continuous integration and continuous deployment (CI/CD) activities are practiced in companies (Ståhl and Bosch 2014). These activities are central to

---

✉ Aymeric Hemon  
aymeric.hemon-hildgen@essca.fr

Barbara Lyonnet  
barbara.lyonnet@univ-nantes.fr

Frantz Rowe  
frantz.rowe@univ-nantes.fr

Brian Fitzgerald  
bf@lero.ie

<sup>1</sup> ESSCA School of Management, 1 Rue Joseph Lakanal, 49000 Angers, France

<sup>2</sup> LEMNA, IAE de Nantes, Chemin de la Censive du Tertre, Bâtiment Erdre, University of Nantes, 44322 Nantes, France

<sup>3</sup> SKEMA Business School, KTO, Sophia Antipolis, France

<sup>4</sup> LERO, University of Limerick, Castletroy, Limerick V94 T9PX, Ireland

DevOps. Emerging as it did from practice, there is not a great deal of work thus far outlining the conceptual or theoretical underpinnings of DevOps. In fact, many definitions of DevOps exist, and some aspects of DevOps are quite old and well-known whereas others are new (Hüttermann 2012). A similar situation occurred in the case of agile methods, which was also practice-led, and some important definitional work appeared later (Conboy and Coyle 2009). We believe that many companies are in the transition from agile to DevOps, with different practices being incorporated as they make this transition—continuous integration and continuous delivery, for example (Balalaie et al. 2016).

The inherent innovation in DevOps is the connection between two organizational cohorts, i.e., Development and Operations. The traditional disconnect between these functions led to damaging silos which resulted in major inefficiencies in delivering software. However, connecting the Development and Operations functions has led to the creation of new collaboration patterns among roles including Developers, Architects, Scrum Masters, Product Owners, Release Engineers, Testers. This departs from the traditional collaborations which are well understood and catered for in Scrum ceremonies such as the Daily Standup, Backlog Grooming, Sprint Review/Retrospective. In this context, we analyze five key job roles in a company engaged in DevOps (Release Manager, Architect, Product Owner, Department/Project Manager and Production Engineer) and describe the key collaborations across these and other DevOps roles. We then describe the skills for each of these roles, dividing them into ‘hard’ and ‘soft’ skills (Gallivan et al. 2004; Robles 2012; Wong et al. 2006). As new technology, methods and approaches evolve “employers expect their employees to add more and more skills to their repertoire” (Goles et al. 2008). We consider how soft skills (SSk) vary by level of automation.

Given the above, our research had the following objectives:

1. To understand the importance of skills and collaboration patterns in the progression towards DevOps.
2. To discuss the extent to which progression towards DevOps reflects greater smartness

We augmented our conceptualization in relation to the above with an empirical study of a large multinational company which allowed us to illustrate our research objectives in a very tangible way.

This paper is structured as follows. Section 2 discusses related research on the core topic of smartness, the transition to DevOps, the skills and competences that are necessary in a DevOps context, and the collaboration patterns among the roles that comprise software development. Section 3 provides details of our case study research method in a large multinational company. We present our findings in Section 4 and discuss implications in Section 5. We conclude with limitations and possible future research in Section 6.

## 2 Background and Related Work

### 2.1 Agile, DevOps and Level of Automation of Design and Flows

Agility is defined as “the continual readiness of an entity to rapidly or inherently, proactively or reactively, embrace change, through high quality, simplistic, economical components and relationships with its environment” (Conboy and Fitzgerald 2004). Agile methods arise from the inability of conventional methods (e.g., Waterfall) to cope satisfactorily with a changing environment, and to ensure the delivery of business value in high quality systems without incurring significant delays (Highsmith and Cockburn 2001). DevOps in turn emerged to address the bottleneck which was preventing Development teams from delivering to Operations in a faster and more frequent manner. However, agile methods form the basis of DevOps. The Agile philosophy has an impact on management styles, collaboration, control, new skills set, training or recruitment (Conboy et al. 2011; Nerur et al. 2005). The DevOps approach requires a strong and continuous connection between Development and Operations teams (Fitzgerald and Stol 2017) DevOps fosters the creation of cross-functional teams where each team member needs to consider and anticipate the job to be done by other members. For example, developers need to understand the real-world production environment where their colleagues release the code. In the same way, Operations need to operate in a manner compatible with how Developers produce the code, test it, and build the delivery package. It is therefore fundamental in a DevOps configuration to increase test automation (Lwakatare et al. 2015) to optimize end-to-end deployment processes. Indeed, “*automation is the key to efficient collaboration and tight integration between development and operations.*” (Wettinger et al. 2014). Humble and Molesky (2011) presented four core values for DevOps: Culture, Automation, Measurement and Sharing. In the recent DevOps handbook, (Kim et al. 2016) also identify the importance of culture, suggesting “*a high-trust culture that enables all departments to work together effectively, where all work is transparently prioritized and there is sufficient slack in the system to allow high-priority work to be completed quickly.*”

The increased adoption of continuous integration, continuous delivery, and DevOps practices underlies the importance of closely linking software development and its physical implementation with its supporting and accompanying infrastructure. This is evident in the suggestion that we view infrastructure as code (Forrester Research 2015). Historically, hardware and software systems have been manually configured by individuals or teams of operations engineers. However, as the number and complexity of these machines has increased, there is growing interest in automating the scripts required to configure and maintain these machines. With typical server farms now comprising many thousands of machines, problems caused by configuration drift or

unauthorized changes can result in considerable disruption and extremely costly downtime.

Treating infrastructure as code provides several benefits, including the ability to have infrastructure code that is automated, versioned, and applied repeatedly and consistently. Several new tools have emerged to support this, including Chef,<sup>1</sup> Docker,<sup>2</sup> Puppet,<sup>3</sup> and Vagrant.<sup>4</sup> These tools further enable the closing of the gap that exists between developers and operations staff (Humble and Molesky 2011). The level of automation of design and flows is thus reflected in the move towards infrastructure as code and continuous delivery.

## 2.2 Being smarter with DevOps

Alter (2018) proposes a useful definition of smartness in relation to socio-technical systems:

“a purposefully designed entity X is smart to extent to which it performs and controls functions that produce directly perceptible results for people by using automated capabilities for processing information, interpreting information, and/or learning from information that may or may not be specified by its designers.”

We use this definition to consider the extent to which DevOps can be classified as smart. Firstly, DevOps arose as a deliberate and purposeful attempt to bridge the gap between the Dev and Ops function, so it clearly complies with the *purposefully designed* element of the above definition. Furthermore, DevOps seeks to *perform and control functions* in that it seeks to control how two traditionally separate functions Dev and Ops operate as one. The results are *directly perceptible* in that they are evaluated by cross-functional teams that act based on the output from the process, for example, sprint burndown charts, sprint board etc. In relation to *automated capabilities*, this is at the heart of DevOps. Continuous Integration builds are automatically triggered by code commits to a repository. Unit and integration testing is automated typically by a range of tools designed specifically. *Information processing* is very evident in the display of information, as in the sprint boards etc. Also, it is quite common in DevOps environments to have a physical display which indicates a broken build, a traffic light showing red for example.

Alter (2018) also suggests that smartness in socio-technical systems is context-dependent. We would argue that DevOps teams are also very context-dependent. In teams using Continuous Delivery in DevOps, the capture and the use of the information is directly put into context. This context-related knowledge is a basis of DevOps system to ensure this continuous and effective communication between Dev and Ops.

<sup>1</sup> <https://www.chef.io/chef/>

<sup>2</sup> <https://www.docker.com/>

<sup>3</sup> <https://puppetlabs.com/>

<sup>4</sup> <https://www.vagrantup.com/>

## 2.3 Skills and Competences of IT Jobs

“A skill is a combination of ability, knowledge and experience that enables a person to do something well.” (Boyatzis and Kolb 1995). Skills are specific to a domain and developed through practice (Boyatzis and Kolb 1995). In general, the literature divides IT-related skills into technical and non-technical skills (Goles et al. 2008), and also into ‘hard’ skills and ‘soft’ skills or behavioral skills (Gallagher et al. 2010; Gallivan et al. 2004; Maram et al. 2009; Robles 2012). Non-technical skills are sometimes divided into business, management and inter-personal skills (Goles et al. 2008).

Firstly, technical or hard skills refer to the ability to perform, to do something well, using technical knowledge, expertise, techniques, practices. (Maram et al. 2009) propose a set of technical skills which include knowledge of programming language concepts, scripting, code optimization, testing and debugging, adherence to standards, security, frameworks, integrated development environments (IDEs), data structures and databases, operating systems. We drew on this categorization as these skills are certainly relevant to developers who must know about the art of programming in general, how to test, debug and fix problems, and also to write scripts using a specific language as Java, Python, C++ or PHP. However, we augmented with hard technical skills more relevant to the Ops function, i.e., for a Production Engineer or a Systems Administrator, hard/technical skills would include the use of specific tools for monitoring and log analysis (e.g., Splunk), orchestration (e.g., Rancher), containerization (e.g., Docker), or automated configuration management (e.g., Chef, Puppet). Finally, in addition to the technical skills listed above, we considered management skills as hard skills because they are based on specific technical knowledge, i.e. planning, time monitoring, etc., and on the use of particular tools (Tables 1 and 2).

Both hard and soft skills are necessary for a successful outcome. In a study of software engineering management at Google, (Garvin 2013) highlighted the connection between the technical and management skills, concluding that “*it’s a delicate balancing act to keep employees happy and motivated through enthusiastic cheerleading while helping them grow through stretch assignments and carefully modulated feedback. When the process works well, it can yield extraordinary results.*” An interesting study at Microsoft revealed that great managers were perceived as those with an acceptable level of technical skills but outstanding soft skills, rather than the reverse of acceptable soft skills and outstanding technical skills (Kalliamvakou et al. 2017). It is better to have an IT manager with soft skills, i.e. inspiring the team, supporting experimentation, facilitating communication, than to have an expert or the most technical person within the team. Consequently, soft skills play an important role for IT job profiles.

**Table 1** Hard Skills (HSk) adapted and extended from (Maram et al. 2009)

- Systems (SYS) – Programming Language Concepts, Data Structures, Operating Systems, Relational Database Management Systems, Hardware and Infrastructures.
- Coding and Programming Languages (CPL) – java, javascript, C++, PHP, etc.
- Software Engineering (SWE) – Software Engineering – Life Cycles, Software Engineering – Models, Agile Methods, DevOps approach.
- Architecture (ARC) – Information Systems Urbanization, Technical Architecture, Software architecture, Software Design, UX Design.
- Testing (TST) – Unit testing, Integration testing, Component interface testing, System testing, Operational acceptance testing.
- Quality, Security (QSY) – Company Quality Management Systems, Supervision, Monitoring, Orchestration, Maintenance.
- Release Management (RMT) – Release, Deployment, Provisioning, Artefact Management.
- Management (MNG) – Team Management, Planning, Budgeting, Time monitoring, Reporting, etc.
- Negotiation (NEG) – Proposing, Trading, Negotiation, Bargaining, Compromising.

## 2.4 Roles, Skills and Competencies of IT Jobs in Transition to DevOps

### 2.4.1 Roles

In this paper, we focus on the jobs which are a priori most impacted by DevOps in terms of skills and competencies through their often-multiple roles. In consultation with industry experts, we selected five jobs of core interest: 1) Product Owner, 2) Manager, 3) Architect, 4) Production Engineer who performs the tests as Ops (in fact developers also perform tests in a DevOps mode), 5) Release Manager. We did not consider developers in this paper. Although developers are certainly also impacted in the way they share knowledge and collaborate, we did not see any particularly significant challenges for them in moving towards DevOps. Our main rationale is that the core activity for developers is coding and that while they would have to consider additional factors when coding in a DevOps context, the coding role would still be central, and they would not have to evolve as much as others.

The **Product Owner (PO)** role is essential when working in agile with Scrum. POs play a dual role, as representative of the client needs, but also with a real operational role that links the business to project management. POs are responsible for optimizing the value of what development teams produce. Autonomy is essential for POs to succeed and their decisions

must be respected by all stakeholders (Coyle et al. 2015; Schwaber and Sutherland 2011). There is some debate as to whether POs can also act as project managers (PM). They can definitely be managers as indicated in the Scrum guide, “Product Owner is the sole person responsible for managing the Product Backlog” (Schwaber and Sutherland 2011). However, there is no clear indication regarding project management in the Scrum method. The Scrum guide mentions that “Scrum Teams are self-organizing and cross-functional. Self-organizing teams choose how best to accomplish their work, rather than being directed by others outside the team” and “Scrum recognizes no titles for Development Team members other than Developer, regardless of the work being performed by the person” (Schwaber and Sutherland 2011). Therefore, this means implicitly that self-organization is compulsory which avoids the possibility for the team to be managed by a PM from the team or external to the team. Theoretically in the Scrum method, POs are managers, but not conventional project managers. POs are associated with business ownership rather than project management even if they are accountable for the product backlog management and the final product delivery. However, in reality a PO can also assume the function of PM. Beyond, the distinction between the PO and the PM, some studies mix the PM profile with Scrum Master (SM) profile to identify Scrum Product Owner competences (Oomen et al. 2017).

**Table 2** Soft Skills (SSk) adapted from (Robles 2012)

- Communication (COM) – oral, speaking capability, written, presenting, listening.
- Courtesy (CSY) – manners, etiquette, business etiquette, gracious, respectful.
- Flexibility (FLX) – adaptability, willing to change, lifelong learner, accepts new things, adjusts, teachable.
- Integrity (ITG) – honest, ethical, high morals, has personal values, does what’s right.
- Interpersonal Skills (IPS) – nice, personable, sense of humor, friendly, nurturing, empathetic, has self-control, patient, sociability, warmth, social skills.
- Positive attitude (PAT) – optimistic, enthusiastic, encouraging, happy, confident.
- Professionalism (PRO) – businesslike, well-dressed, appearance, poised.
- Responsibility (RES) – accountable, reliable, gets the job done, resourceful, self-disciplined, wants to do well, conscientious, common sense.
- Teamwork (TWK) – cooperative, gets along with others, agreeable, supportive, helpful.
- Work Ethic (WKE) – hard working, loyal, initiative, self-motivated, on time/attendance.



The role of **Manager (DPM** for Department & Project Manager) is a significant one, and has multiple definitions in the literature. In our conception, a manager is a person with the responsibility to deal with resources, tangible and intangible, to serve a specific objective. The Managers may be project leaders or simply team leaders, or both. It is simply a feature of the hierarchy that often places these two roles in the same person. Others like SM are either not always present or are played by the more traditional role of developer or PM. Team Managers (TM) include different professions: development/operational or supervisory team leader, and qualification-integration manager. PMs define and manage an IT project from conception to delivery to get an optimal result for customer requirements of quality, performance, cost, time and security. The **architect (AR)** job is evolving. The AR is often able to develop multiple skills, whether he or she plays a technical AR, a software-application AR, or a functional AR role. On the Ops side, **Production Engineers (PE)** or production integrators and testers are responsible for production, operations, incident monitoring, support and user support. PEs participate in the development of architectural files and the production of applications. PEs also provide the expertise and support for incident resolution. Finally, on the Ops side, the **Release Manager (RM)** is key to ensuring the success of projects. RMs are responsible for the deployment processes. They follow the different versions and coordinate between the development and test teams and the deployment teams. The RM is associated, in the firm's context, with the function of PM Implementation. This is an essential activity, but the scope is questioned by DevOps. We investigated to what extent SSk were perceived as important by these five roles when transitioning from agile to DevOps and the implications for collaboration.

#### 2.4.2 Skills and Competencies

Gallivan et al. (2004) identify a “long history of IT-skills studies” which suggest that soft or non-technical skills are sometimes more important than technical skills. Wong et al. (2006) also emphasize the relative importance of SSk – flexibility, adaptability, motivation and effective communication. Non-technical skills include interpersonal, leadership, organization, independence/motivation, and creativity skills. However, studies found that non-technical skills were far less valued than technical skills in recruitment advertisements. They concluded that despite the emphasis on hiring well-rounded employees with good business knowledge and SSk, the recruitment process focuses on “hard skills” because they were easier to screen. Thus, they confirmed the “recruitment gap” that had been identified earlier (Todd et al. 1995; Trauth et al. 1993). This lack of recognition is harmful as some SSk have been identified as important in agile software teams (Vivian et al. 2015). In addition, Wiedemann and Schulz

(2017) investigated key capabilities of DevOps teams which could foster competitive advantage. They identified seven key capabilities (Change Readiness, Decision Making, Culture, Collaboration, Intrapreneurship Skills, Agile Project Management, IT Technical Skills, Continuous Skills). However, they mixed capabilities with skills without determining precisely which skills were a source of competitive advantage. In their study, they suggested some ways to foster collaboration within DevOps team but did not specify the extent of such collaboration. In a companion study focusing on new forms of collaboration, (Wiedemann 2017) highlighted assimilation stages of innovation within DevOps teams but did not consider the question of the extent of the collaboration.

Therefore, and beyond the growing interest in skills and collaborations, the literature suggests that there might be different skill sets (hard and soft) as well as different collaborations depending on a level of automation when transitioning to DevOps.

#### 2.5 Collaboration Patterns

As skills can evolve in the transition towards DevOps, collaboration patterns may change. In terms of soft skills, teamwork evokes cooperative and supportive behavior which is related to collaborative capability. Collaboration patterns address the social network staff use when performing their work (Nerkar and Paruchuri 2005). As in any activity, in software engineering developers generally specialize in a job (with potentially multiple roles) and turn to team members with complementary competencies to fill the necessary tasks for these roles. Going through more rapid iterative development cycles requires members to be more polyvalent, even if they keep their core competencies and job definition. However, when transitioning to DevOps we might foresee also a change in networks of specialties that people need to access to on a regular basis. When going from agile to DevOps there is supposedly a widening of the team. In fact, instead of having two teams interacting through their managers (only), all members can potentially interact directly and know who is doing what. However ultimately and as automation progresses, a DevOps team may not encompass many different jobs.

Knowledge sharing and coordination in collaboration may also change with DevOps (Humble and Molesky 2011). By definition DevOps is based on sharing practices and building a common ground and bridging different cultures. While in agile people interact much more often than in plan-driven methods this is limited to Roles from the development world. Those already share parts of a same culture where values such as speed of delivery, creativity and innovation prevail. Agile practices can be interpreted as coordination mechanisms which facilitate knowledge sharing among team members (Strode 2016). Nevertheless, while teamwork becomes important in agile as opposed to plan-driven, individualism and

competition may also prevail. Most importantly in agile mode we may hypothesize that Dev and Ops people really work separately and coordinate only very occasionally. We could say that acting in separate functions, they are not sharing their mental models. On the contrary, in a DevOps mode, we argue that even though some tasks may be done separately, mental models or at least respective constraints are much more shared during activities, so that, by drawing a parallel with Bruns' (2013) theorization of coordination in collaboration of R&D teams for which she refers to “working alone together”, even when separated, people work “separated together”. In addition, joint work activities with Ops are much more numerous in DevOps.

### 3 Methodology

Our first objective can be translated into two research questions. How do individuals collaborate in teams and what skills do they need? Do patterns of skills and collaboration vary across teams with different degrees of automation? Whereas the first question requires analysis at an individual level, the second question is at team level of analysis.

We followed the case study method combining interviews, observations and documentation to explore and analyze a new phenomenon: DevOps. A case study is an empirical enquiry that investigate a contemporary phenomenon within its real-life context, especially when the boundaries between phenomenon and context are not clearly evident (Yin 1994, p. 13).

In order to study how automation influences skills and collaboration we study 12 teams who develop apps in the same context of a large European IT services firm with 15,000 of IT Staff. It is a historical company on the market. The company is structured with seven hierarchical levels and can be characterized by a very bureaucratic culture, which runs against agility (Strode et al. 2009). The Information Systems Division is divided into two functions, the development and the exploitation departments. In this company employees have been using agile methods since more than fifteen years and DevOps for eight years, hence being one of the early adopters of the DevOps approach.

The 12 teams are embedded in a firm with some similar conditions (all use agile management methods) and we study five job roles as units of analysis (Runeson and Höst 2008). However, these teams use different agile engineering methods, so their infrastructure and degree of tests automation are different. We reason that, as they move towards DevOps, automation will affect the human skills that may be required and collaboration patterns (how and with whom job roles will collaborate).

This study was realized over 12 months, in 2017. We studied 5 job roles and their scope of collaboration in 12 teams, which involved 59 in-depth face to face individual interviews (these were recorded and lasted 90 min on average). About

DPM, we interviewed six PM and six TM. Interviews were conducted by three interviewers and were subsequently transcribed and coded. Data was analyzed following the six steps of thematic analysis (Braun and Clarke 2006). Themes emerged from codes. We applied theme “keyness” rule where a theme “is not necessarily dependent on quantifiable measures – but in terms of whether it captures something important in relation to the overall research question” (Braun and Clarke 2006, p. 10). Prevalence of themes was counted following two methods: at the level of the data when a theme appeared anywhere in each individual interview; at the individual level when the interviewee evoked a theme. To ensure reliability, we respected data’s four criteria of trustworthiness (Lincoln and Guba 1985). Finally, validity was checked by returning to the participants with interpretations for approval.

Our inductive research study comprised two phases:

In Phase 1, we first conducted a review of the literature to identify appropriate criteria for selecting the sample of 12 teams. Second, we observed and interviewed four teams for 15 days (CI and CD) to compare elements in the literature and those in practice, i.e. size of the team, automation level (as per our model), and level of externalization. Third, we conducted a series of interviews with 12 strategists in IS and Human Resources to incorporate their vision and to validate the selected sample.

In Phase 2, we used three criteria to select the sample of 12 teams.

1. the size of the team measured in terms of number of staffs assigned to the project with small team equal to or less than 14 people and large team larger than 15 people;
2. the outsourcing policy and more precisely the level and nature of outsourcing on the project, e.g. we realized that fixed-price contracts can pose specific problems when working in agile mode. We contrasted two situations: some project activities carried out in work package and fixed price mode, and project without outsourcing or with an outsourced technical assistance contract;
3. the level of automation of flows between development and operations functions which stem from our discussion of infrastructure as code in section 2.1 above.

In relation to team size, it should be noted that there is by no means universal agreement as to the actual definition of a large project (Rolland et al. 2016). Proposed measures include the number of development teams (Dingsøyr et al. 2014), the number of developers (Elshamy and Elssamadisy 2007; Hannay and Benestad 2010), the number of lines of code (Petersen and Wohlin 2009), and the duration of the project (Bjarnason et al. 2011). Even in the case where the measure involves the number of developers, there is no agreement. Hannay and Benestad (2010) suggest that more than 20 developers could be viewed as large, whereas Elshamy and Elssamadisy (2007) suggest that 50–100 developers is a large

project. In our case, we discussed the issue with the relevant manager, and it was suggested that when teams exceed 12–15 members, the differences due to largeness become apparent. Hence, we chose 14 as the cut-off figure in our study. In relation to outsourcing, projects that are outsourced demand greater soft skills (e.g. Communication) (Goles et al. 2008).

We identify three automation levels:

1. **Automation Level 1 Agile (Aut.1):** Compared to the traditional V-model, agile can be considered as automation level 1 in the transition to DevOps, because it facilitates more frequent releases as development becomes more iterative. However, DevOps is not realized because Dev and Ops continue to work in silos with limited sharing, nor automation of releases.
2. **Automation level 2 Continuous Integration (Aut.2):** The Ops function needs to be aligned with the Dev function, and both begin to perform various tests (unit and non-regression tests) (Ståhl and Bosch 2014), which are synchronized with code development, and also automated as far as possible.
3. **Automation level 3: Continuous Delivery Stage (Aut.3):** Integration tests with other components, end-to-end tests, performance tests, user acceptance tests are then are co-designed, performed and preferably automated by Ops in conjunction with the Dev function (Chen 2017). The greater the level of automation, the more it requires sharing and mutual discussion about measurement, so that the right indicators can be contextually designed to monitor the software development activity.

All possible configurations allowed us to explore 12 distinct types of teams. This even distribution of teams for each of the three criteria allowed us to explore the effect of the level of automation while observing a diversity of teams in terms of the two other criteria. No previous research had studied the effects of the level of automation on skills and collaboration in the DevOps context (Table 3).

Documents, observations and interviews performed in the Phase 1 considerably helped us in interpretation of the 59 interviews in the Phase 2. The scope of collaboration scope was established by requesting interviewees to identify those with whom they worked most. While this is susceptible to bias, we did not want to induce other biases by repeating this question for every possible job role they could have collaborated with. Collaboration enrichment was examined through responses to whether and how individuals were satisfied or unsatisfied in our semi-structured interview (Hemon et al. 2018). To investigate the skills issue in the 59 interviews, we asked what skills (know-how, behavior) were necessary for the evolution of their jobs. We then analyzed interviews and compared our results to Maram et al. (2009) and Robles (2012) sets of Hard and Soft skills (cf. Tables 1 and 2).

**Table 3** Distribution of 59 interviews per contingencies

|                                                                                       |
|---------------------------------------------------------------------------------------|
| Team Size: 30 Small (min: 5; max: 14) / 29 Large (min: 15; max: 80).                  |
| Type and Level of Sourcing: 30 Outsourced Fixed-Price / 29 Other (Internal or other). |
| Automation Level: Aut.1: 19 / Aut.2: 20 / Aut.3: 20.                                  |

4 Findings

4.1 Hard Skills and DevOps Maturity

All nine groups of Hard skills (HSk) adapted from Maram et al. (2009) were identified by the 59 interviewees. The results of the verbatim analysis show a representation of HSk in accordance with the literature. Typically, DPM and PO focus easily on managerial skills (MNG) while PE favor highly operational technical skills (TST, QSY). We did not find any automation level effect on hard skills except for the lower managerial skills as automation level increases.

4.1.1 Importance of Technical Skills Regardless of the Level of Automation

One of the issues emerging from our interviews is the systematic reference to hard skills, regardless of the level of automation. However, for PE, and regardless of the level of automation, it appears that technical skills overwhelm all other skills. PE, for example, favor the skills of Testing (TST) and Quality, Security (QSY). In addition, we noted that the more the automation is, the more precise the PEs are in what they say about TST skills. A PE (Aut.1) declares “It is a challenge (...) to work so that the automatisms of tests are good, to make the processes of delivery (...) good”. Then, a PE (Aut.2) highlights the importance of “automatic testing tools, automatic installation tools, and then operations, the tools of orchestration, supervision”. Finally, a PE (Aut.3) states “There is no need to be an expert in software development, it is not what is required, but you must have a culture of code (...) to know that when we will produce a little bit of code, we have to version it, it must be referenced and tested, etc.; it’s the basics. Then, about skills, you need to have a little bit of security, automation skills”.

4.1.2 A Representation of Management (MNG) Skills Evolving in Accordance with Automation Level

DPM spotlight skills in MNG and then more technical skills such as Software Engineering (SWE). On the other hand, we note that the higher the automation level, the less the MNG skills are mentioned by the interviewees. It seems that this is due, on the one hand, to an automation level effect which pushes the teams to self-organize, and, on the other hand, to a release of the traditional management teams and the organization facilitated by the DevOps philosophy. At automation level 1, DPMs can feel it.

A DPM (Aut.1) affirms “Since you will witness the project, the delegation and the autonomy of the people, it is a very important criterion”. Another DPM (Aut.1) confirms: “If they (Ed.: developers) are self-organized, they must stick their neck out. They take a subject, they go to the end, they bring their ideas, they apply them and then when they fail they repair. We must assume (...) You have to know them to put them in situation and to motivate them on this or that subject”. At automation level 3, and except for one DPM in a large team, none of the 12 DPMs mention MNG skills.

We found a similar effect, but less marked, on the NEG skills that are less emphasized as automation level increases. NEG skills are less evoked and mobilized in high automation level; this could be partly explained by the fact that the DevOps philosophy requires more frequent and sustained collaboration between the different actors, which facilitates an adhesion and an alignment of the actors. Many stakeholders at automation level 1 emphasize the importance of negotiation in their activities and hence on NEG skills. A PO (Aut.1) states: “especially in the steering committees we have 3 large parties that do not necessarily have the same objectives or in any case that do not have the same issues. We have to identify them, put them on the table and find the negotiation that makes one accept, the other too.” A RM (Aut.1) confirms: “A great sense of negotiation, you need to have that.” However, one PO (Aut.1) stands out by claiming “all this sharing (...), with the communication, (...) to exchange around that, to finally share. It seems to me to be relatively essential so that everyone is aligned, that we all look in the same direction.” This alignment therefore limits negotiation. This vision is widely shared at automation level 3. A DPM (Aut.3) testifies: “it’s important to understand the other jobs. Because if we understand the other professions, we will analyze better (...) if we want to work in an integrated team or Agile, or go really in that direction, there is a need that all the members of the team understand the expectations of others. That it is of those who express their needs in terms of trade, that those who will develop it, that those who will exploit it. And if one manages to do that, it means that at all the levels one will be able to work better in common. Because when we do not understand each other’s, it inevitably creates tensions behind”. Another DPM (Aut.3) confirms this need to seek alignment making things happen: “my role is to be really available and responsive to the expectations of the different stakeholders. That’s really something that is very addictive.”

## 4.2 Changes in Soft Skills Perceptions when Considering Automation Levels

Codification of interviewee skill perceptions allowed us to complete nine of the ten groups of SSk identified by Robles (2012). Only half of Robles’ 10 groups of SSk are mentioned by interviewees for each automation level. Five groups of SSk are under-represented if not absent within representations of Agile and DevOps teams whatever the automation level, i.e.

ITG, CSY, PAT, PRO and WKE. Among them, positive attitude (PAT) clusters various skills, i.e. being optimistic, enthusiastic, confident, encouraging, and is only cited by PO and DPM at higher level of automation (2 and 3). Beyond these results, we came across five groups of SSk represented in each automation level (COM, IPS, FLX, TWK and RES).

Communication skills (COM) refer to the ability to communicate both orally and in writing, presenting and listening, and are present at each automation level. A RM highlighted the importance of COM: “I think you have to know how to communicate” (RM, Aut.1). This perception is shared by many people on both sides, Dev and Ops, e.g. a PO (Aut.2): “You have to listen, so especially not having the posture of “I know, I have experience“, especially today”. A DPM (Aut.3) confirms the central position of communication saying that “Communication is still a fundamental basis of teamwork. And I think that often, either because we are partitioned (...) we are somewhat inclined to understand or integrate neither the constraints of the rest of the team nor other people from other teams”. We noted that the perception of COM was slightly higher in terms of diversity and frequency of mentioned skills when automation level was going up. These previous verbatim show COM at Aut.1 covering basic COM principles as to convey an information between two or more stakeholders, then COM at Aut.2 with a stress on listening skills integrating humility, and finally COM at Aut.3 with a wider coverage and a higher level of COM incorporating large understanding skills and their usage to interact with teams and integrate their constraints.

Interpersonal skills (IPS) are related to sociability, empathy, nurturing, friendly, patience; self-control capacity. As for COM, IPS appear at each automation level. A PO underlined the importance of IPS to satisfy different stakeholders and reach final goal: “Coordination, of course, then relational, because everyone is not going necessarily in the same direction or does not necessarily have the same objectives (...) On the one hand, the production and the operations managers ask to limit the number of releases or production launches (...) on the other hand, we have businesses that always ask for more.” (PO, Aut.1), a shared vision by a DPM (Aut.3): “Stakeholders analysis! I change partners very often. So, know how to evaluate a situation quickly, if I have someone who is supportive, or if someone who is perhaps a little more reluctant (...) You are asked to get closer to a certain number of people with whom you are not used to work with (...) so it requires human qualities.”. The perception of IPS was slightly larger in terms of diversity and frequency of skills when automation level was higher.

Flexibility skills (FLX) signify to be ready to change, to learn throughout the life, to accept new things, to adjust, to adapt. FLX are present at all levels of automation and on both Dev and Ops sides. Moreover, people from automation level 3 seem to have a clearer and wider representation of FLX. A PE (Aut.1) simply mentioned one sub-skill of FLX group, adaptability: “Adaptation is perhaps most important, because it changes all the time,



businesses, tools, applications and technologies”. At a similar position, a PE (Aut.3) asserted: “Everything evolves so quickly...it is better to know how to learn than to develop know-how because that will change, it is important to be open to the novelty”. A PO (Aut. 2) stated that: “You have to feel and accept the error. One must accept sometimes and assume one has made a mistake, and it is a step backwards”. The skill is about adaptability but with a deeper and larger interpretation of learning and adaptability. Finally, a DPM (Aut.3) in a small team suggested: “You must be humble or agree that you don’t know...your neighbor knows better and will show you. (It is better) not to impose your vision either.” Meanwhile another DPM (Aut.3) confirmed this “The challenge is to change the subject - just able to juggle these topics. It necessarily requires great flexibility and adaptability. A questioning, because suddenly, you’ve been in the business a long time, have already proven worth in the past, and you are asked to work in another way”. We discovered that FLX perception was broader and much more important in terms of diversity and frequency of mentioned skills when maturity level was higher.

Teamwork skills (TWK) indicate a capacity to cooperate, to be supportive, collaborative, helpful. TWK are intimately linked to Agile Principles, i.e. “Business people and developers must work together daily throughout the project”, as well as Agile Methodologies (Scrum, XP), Agile Practices (daily stand-up, sprint review) or DevOps philosophy breaking silos between Dev and Ops and building bridges and fostering new teams. As mentioned by a PO, it is important to “understand that we do not work each one in his own corner (...) it must be shared a minimum, that everyone knows where the others are (...) all the developers do not need to know where all the integrators are, but we can have a roughly correct and almost real-world view of where we are. to know how to adjust, to know how to project also.” (PO, Aut.1). From the Operations side, an RM highlighted a cultural change regarding TWK: “We do not ask for technical expertise anymore, because we really want people to stop solving the problem technically. (We prefer) to get the problem solved by a community. It is really a different positioning” (RM, Aut.2). A vision confirmed by another Ops: “You really need to have the vision of belonging to the same team, and not to say: “Me, I do represent Production (Ops)“, to feel “powerful“ because we (Ops) take the decision in fine to put the application into production.” (PE, Aut.3). Finally, a DPM in a small team (Aut.3) demonstrated her way to become supportive and collaborative integrating fully her team: “You also have to be ready to join this team even if you are a manager of some of them. We (the DPM) must also know when to take a back seat.” When team members are really acculturated to work together, i.e. in advanced DevOps team, silos disappear. We figured out that the perception of TWK was moderately larger in terms of diversity and frequency of mentioned skills when automation level was higher.

Lastly, as a DPM (Aut.3) stated clearly: “the responsibility necessarily, to know to be responsible.”, thus Responsibility

skills (RES) which covers two aspects. The first one is more evoked by PE: get the job done, and well done, conscientious, accountable. The second one is more perceived by DPM: trustworthy, astute, common sense imply to get the job done, to be conscientious, accountable, reliable. These skills are mostly cited by DPM, afterwards PE and to a lesser extent RM, and then few PO and very few AR. A large and complex PM suggested: “I want to say for commitment too. If they are self-organized, they must get in deep. They take a subject, they go to the end. They bring their ideas, they apply their ideas, and then when they fail, they repair. We must assume.” About conscientiousness, the sense of getting the job well done and being resourceful, a PE in a large team (Aut.3) declared: “You do not have to be an expert to find the solution. You must understand what’s going on. And about the know-how-to-be well, you must listen rather than talk.” We identified this same preoccupation of consciousness, reaching objectives with POs trying to stay the course: “do not do the wind vane. We can be wrong, but on the other hand, we do not return every day on what we said the day before. (...) It is disastrous for the progress of work.” (PO, Aut.1) We detected that the perception of RES was decreasing in terms of diversity and frequency of mentioned skills when automation level was higher.

To conclude, we found that automation level is a factor impacting SSk perception. First, FLX are much higher in team members in a more automated environment (Aut.3). Secondly, the higher the automation level, the more the presence and the diversity of COM and IPS increase. Third, TWK and team spirit, are more present in an Aut.3 team compared to an Aut.2 or 1, which may seem logical but not obvious, particularly when considering the fundamentals of the agile approach which require a Teamwork culture. Fourth, surprisingly, RES are more present in a lower automated context (Aut.1) and decrease when automation is higher. This could mean that in an agile environment, the sense of responsibility is more formally distributed, and in a continuous delivery configuration, the sense of responsibility is more shared within the team. In a DevOps environment, we can explain this result with automation that could reduce and /or transfer responsibilities and therefore responsibility skills.

Type of Outsourcing may have an impact on SSk ‘perceptions. Some competences, such as RES or TWK are more present when the project is internal. In contrast, outsourcing seems to be a factor which results in increasing FLX and IPS. The perception of SSk seems to vary less with team size than with type of outsourcing. Overall maturity level is the most important.

### 4.3 Roles and Collaboration Analysis

To the question about the actors with whom they interacted the most, all interviewees cited more than 15 different co-worker’s functions. To better understand the changes in the pattern of collaborations between the Dev and the Ops workers per automation level, the Table 4 presents in white cell the number of

collaborations cited within Dev only or within Ops only and in grey cell the number of cited cross-functional collaborations within Dev and Ops. For automation level 1, teams, main collaborations between Dev and Ops are less numerous (20) than those within Dev and within Ops (31). Collaborations crossing the two functions become slightly less numerous than those internal either to Dev or to Ops (26 against 29) for teams in automation level 2, and then become balanced (31 against 32) for teams in automation level 3 (See Table 4). At automation level 1, two fifths of collaborations are DevOps Crossing collaborations, while at automation level 3, one out of two collaborations is a DevOps Crossing collaboration.

Firstly, our results indicate that the higher the automation is, the more the collaboration between Dev and Ops intensifies. Secondly, this balancing with DevOps means that collaboration with members of other function are perceived as important as internal collaborations within Dev and Ops functions respectively.

Table 5 lists these main collaborations by role/profession; for example, 4 of the 11 RM reported the developer as a main co-worker. Some similarities and differences in collaborations are observed for the five key interviewed roles. As expected, the DPM reported the highest number of collaboration with different co-workers in comparison to the four other interviewed roles. This highlights their large scope of collaborations. Another important observation is the key role of the PO: all interviewed professions have mentioned their collaboration with the latter.

In Table 5, we can see more precisely the main collaborations between Dev and Ops. From the Dev side, we highlight collaborations between PO and RM, AR with operators or DPM with PE and operators. From the Ops side, we see a strong collaboration between PE and Developers, between RM and PO and with developers. Given the question asked, in traditional developments mode or in agile (Aut.1), these collaborations would not have been considered as main collaborations.

Moreover, all respondents have highlighted an improvement of the collaboration's quality between co-workers in an agile context. As mentioned during the interviews, the setting up of daily meetings or sprint reviews, but also the use of collaboration tools (such as the Mingle) could partly explain this finding. Indeed, the setting up of these daily meetings will rhythm the collaborations in a formal way. However, the frequency of meetings can be perceived as an overwork. For each role, we also describe the different forms of collaboration and types of actors involved.

**Table 4** Collaboration analysis by level of automation

|     | Automation 1 |     | Automation 2 |     | Automation 3 |     |
|-----|--------------|-----|--------------|-----|--------------|-----|
|     | Dev          | Ops | Dev          | Ops | Dev          | Ops |
| Dev | 21           | 11  | 23           | 11  | 25           | 12  |
| Ops | 9            | 10  | 15           | 6   | 19           | 7   |

**Release Manager (RM).** The RM is at the center of numerous collaborations within the company and in particular with the production team. All RMs underlined interactions with the project manager, generally on a weekly basis. This collaboration is, and must be, continuous to prioritize certain tasks and thus ensure the success of projects. For example, a RM explained (Aut.3): “Every week we establish a point called the “Common Work Plan of Exploitability” where we exchange. We go through all actions, all activities in progress, and ... we see what has to be prioritized in the case of new actions”. These collaborations are often real driving forces to create a dynamic for all teams working on the same project. In addition, almost all RMs reported collaborations with production engineers (see Table 5). Their exchanges are frequent, (RM Aut.2): “We discuss certain points between us before going to meet the project manager. Given the slightest problem, one goes to see the other and vice versa. “It should be also noted that in this context, the setting-up of project group meetings provided the opportunity to gather RMs, developers and operators. The automation level in the transition from agile to DevOps could influence the frequency of these collaborations. In fact, “The developers for me are hidden behind the project manager. But this evolves within the most advanced projects in the DevOps approach “(RM Aut.3). Thus, an advanced application level of agile and DevOps methods and concepts might naturally lead to establishing more interactions with developers.

**Product Owner (PO).** As expected, most POs reported collaborations with DPM and developers. Some of these interactions are required for the validation of the sprint during the demos. The automation level in the transition from agile to DevOps has a direct impact on the frequency and quality of these interactions. Thus, a PO suggested that to feel part of an integrated team (Aut.3): “Today we are really an integrated team with marketing, developers.” In the case where several POs participate in the same team, they collaborate with each other daily via email or telephone and during the weekly “sales meeting”. This meeting is also a moment of exchange with the Product Manager and line managers. Moreover, some POs mentioned multiple exchanges with functional architects. In line with the various roles inherent to the PO roles, the majority of POs also reported collaborations with business sponsors or users, albeit not considered main ones. It should be noted that several POs highlighted the limited collaborative exchanges with supervisor-operators (Aut.2): “Those with whom I exchange the least, are probably exploitation”; “I have difficulties to express myself on this point, they (“supervisor-operators”) are more related to developers.”

**Architect (AR).** As in previous findings, the three main co-workers mentioned by the architects were POs, DPMs and developers (see Table 5). Most architects reported interactions with DPMs to share information and views on a project. Collaborations were also reported with development team to exchange information, especially during sprint reviews and demos. An architect stressed the importance of this (Aut.3): “we

**Table 5** Main collaborations reported by each role

|                                       |                                             | PO<br>(N=12) | DPM<br>(N=13) | AR<br>(N=11) | PE<br>(N=12) | RM<br>(N=11) |
|---------------------------------------|---------------------------------------------|--------------|---------------|--------------|--------------|--------------|
| OPS ← Main Collaborations cited → DEV | Product Owner, N                            | 3            | 13            | 11           | 11           | 10           |
|                                       | Department & Project Manager, N             | 10           | 13            | 11           | 12           | 11           |
|                                       | Architect, N                                | 3            | 1             | 1            | 4            | 2            |
|                                       | Scrum Master, N                             |              | 3             |              |              |              |
|                                       | Developer, N                                | 9            | 8             | 8            | 12           | 4            |
|                                       | DevOps animator, N                          |              | 1             |              |              |              |
|                                       | Technical expert, N                         |              | 1             | 1            | 3            | 2            |
|                                       | Quality expert, N                           | 1            |               |              |              |              |
|                                       | Multiple collaborators, N                   |              | 2             | 1            |              |              |
|                                       | Qualifier, N                                |              | 1             |              |              |              |
|                                       | Production Engineer, N                      | 1            | 4             | 2            | 1            | 8            |
|                                       | Administrator Infrastructure<br>Director, N |              |               | 1            |              |              |
|                                       | System Engineer, N                          |              |               |              | 1            |              |
|                                       | Operators, N                                |              | 3             | 2            | 5            | 4            |
|                                       | Release Manager, N                          | 2            | 1             | 6            | 6            |              |

have to be precise on how to proceed, and if feasible, establish a stronger, detailed collaboration with the technical manager”. These exchanges take place regularly. Several architects mentioned information exchange with the RM profession, especially to agree on norms, standards and good practices (Aut.2): “he (RM) is supposed to hand over to his team, to read the technical

architectural file”. An AR (Aut.3) highlighted the impact of the transition to DevOps on his work and collaborating: “it’s an approach allowing better understanding the role of each person involved in the project. In the end, each remains accountable; but decision-making in an architecture problem (...) will be more shared than before (...) thus, problems disappear.”

**Production engineer (PE).** The main collaborations cited by PEs are with the DPMs, POs and developers (see Table 5). Thus, most PEs agree on the existence of a significant mutual collaboration with developers (Aut.2 and 3): “We inform each other, it’s natural...with the developers, we are becoming a unique team: the DevOps one”.

In addition, in their daily activities the DPMs must provide applications to PEs so significant collaboration with shared responsibilities have been reported. However, some difficulties of co-operation have been mentioned, for instance a PE pointed out the following (Aut.2): “I was the one who explained what I expected in terms of documentation, whereas it’s their job.” PEs also discussed their collaborations with RMs. However, the frequency of these collaborations can vary a lot across teams. Indeed, in a context where the RM is entirely dedicated to the team, collaborations are frequent. A PE described this collaboration as follows (Aut.2): “In our organization, the role of RM was created because of administrative importance of our project, it’s useful to lighten our work (...) The role of the RM is to anticipate complex operations, to be sure we do not forget anything”. A PE (Aut.3) explained his view of the evolution of collaboration in a team: “The roles of DPM and RM no longer exist in a DevOps team; At the development team level, the role of DPM is operated by the Scrum Master; at the planning level, project management role is a crucial role covered by the team. And the role of RM is operated both by PO, technical expert, and production engineers, and operators”.

**Department and Project Managers (DPM).** DPMs are at the center of multiple collaborations, both inter and intra-team. The manager profile collaborates with many actors (see Table 5). Most also mentioned collaborations with other DPM and POs, in particular to discuss budget, tracking, prioritization of items. Several Managers also highlighted a close relationship with functional architects (Aut.2): “I work early with our functional architects to define and schedule future development.” In addition, although the DPM is not a functional expert, he approves the conformity of the result.

To conclude, the collaborative scope within the main collaborators clearly changed and is now more balanced. The perceived greater richness of collaboration reflects a better understanding among members. This greater richness appears to be more salient when teams are in automation 3. Consequently, considering our findings on skills and collaboration, we could describe three stages in the agile to DevOps transition as follow. At automation level 1, there is a lower range of collaborations which could be explained by the DevOps philosophy itself opening collaboration to a new extent between Dev and Ops while Agile limits collaboration among Dev. Because agile methodologies ask for it, COM skills are very important and RES skills very developed as IPS. However, surprisingly, FLX and TWK are less present in their discourses. At automation level 2, the extent of collaboration is thus wider than for automation level 1. All the SSk are more represented at this level, except for RES which are decreasing. At automation level 3, the extent of collaboration is similar to automation level 2 or can slightly decrease. We explain this finding because when teams reach automation level 3, they know better that anyone with who to collaborate to be more efficient, hence they limit or cancel collaboration with some stakeholders. SSk increase in comparison to Aut.2, except for RES, and this is particularly true for FLX, TWK and IPS.

## 5 Discussion

We summarize our main findings and empirical contributions in Table 6. We discuss these findings and develop theoretical interpretations. In view of these elements, we discuss implications for identifying different stages in the agile to DevOps transition which may lead to the development of a maturity model for DevOps. Finally we propose to explore the implications of our findings in relation to smartness.

**Table 6** Main findings and empirical contributions

| Topic          | Our findings                                                                                                                                                                          | Related work                                                                                                                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Collaborations | Collaboration patterns among the main players have drastically changed and is now more balanced between Dev and Ops                                                                   | “DevOps is something which is present in every organization that employs development and operations personnel. But the amount of interaction differs and depends on the particular organization.” (Erich et al. 2017) |
|                | The greater level of automation in DevOps is associated with an increased frequency of collaborations (daily basis) and with an improvement of the efficiency of these collaborations | The adoption of more disciplined agile and collaborative DevOps leads to a successful enterprise implementation (Ambler and Lines 2012)                                                                               |
| Skills         | The greater the level of automation in DevOps, the more soft skills are required                                                                                                      | Non-technical skills are especially important in agile software development (Gren et al. 2018; Wong et al. 2006)                                                                                                      |
|                | Flexibility and Interpersonal Skills, Teamwork are more important in DevOps than agile                                                                                                | Flexibility and interpersonal skills are important for agile teams (Vivian et al. 2015; A. M. Wiedemann and Schulz 2017)                                                                                              |



## 5.1 Collaboration Patterns

This paper sought to investigate how collaboration changes between Dev and Ops when a company engages in agile and DevOps concepts. Regarding collaboration, we discovered that the collaborative scope among the main participants has drastically changed and is now more balanced. This result is consistent with the fact that team members must collaborate on a daily basis within a single team. In a similar vein, Ambler and Lines (2012) have highlighted the need for a more disciplined agile methodology in an enterprise with a rigorous focus on release, deployment and collaboration between teams. The adoption of more disciplined agile and collaborative DevOps leads to a successful enterprise implementation (Ambler and Lines 2012). We provide greater clarity about collaborations within a DevOps team than previously available (e.g. Wiedemann 2017). Our findings align with Humble and Molesky (2011) who highlighted the importance of collaboration and shared responsibilities and more specifically with the skills, experience and mindset of Ops people. In this context, we observed that POs and Department Project Managers are most consulted, and that collaborative scope among the main collaborators can be slightly enlarged. In teams with a high automation level, we expected less frequent interactions with more diverse participants, however this was not found in our study. In addition, another actor, the Operator who performs supervision tasks, was rarely cited as a main collaborator. This could be explained by the involvement of the Operator in many teams in parallel with no particular focus on a specific team or with specific co-worker. Overall, the perceived greater richness of collaboration reflects a better understanding of who does what in the overall team. Indeed, the transition from agile to DevOps was associated with an increased frequency of collaborations but also with an improvement in the efficiency of these collaborations. Accordingly, the expression of greater richness seems more salient when teams are in continuous delivery (Aut.3). These findings are congruent with Holmström Olsson et al. (2012) research on barriers faced by firms when moving towards continuous deployment. When progressing to continuous deployment from continuous integration, Holmström Olsson et al. (2012) highlighted the need for greater richness and closer collaboration through specific organizational units integrating product management and being fully involved, coupled to proactive customers in order to facilitate closer working.

## 5.2 Shortage of Related Skills as a Barrier to DevOps

Several studies in software engineering have focused on skills (Goles et al. 2008; Gren et al. 2018; Turley and Bieman 1994). Many suggest that non-technical skills are especially important in agile software development (Wong et al. 2006; Gren et al. 2018). However, we are not aware of any studies that have focused on the explicit connection between agile, DevOps and skills. In this paper, we showed that DevOps and Agile methods impact

collaboration and existing IT skills. This result is consistent with research that shows a relationship between performance and skills in software engineering (e.g., Gren et al. 2018). Therefore, it is essential to understand the requirement for new skills in the workplace (Tan and Teo 2007). Furthermore, it is also necessary to understand the requirement for new skills in the curricula for IT students (Bang et al. 2013; Betz et al. 2016).

Companies struggle to determine the most important tasks for IT teams, specifically future Release/DevOps engineers (Kerzazi and Adams 2016). Understanding the evolution of IT skills would help organizations to hire skills that fit their needs (Trauth et al. 1993). This is lacking in DevOps job advertisements which largely neglect soft skills (Hussain et al. 2017) and focus on hard skills, e.g. technical skills, thus confirming a recruitment gap (Todd et al. 1995; Trauth et al. 1993).

Our study provides compelling evidence that the move from Agile to DevOps has an impact on perceived skills in use or in making. While all agile software teams require significant SSk, these are even greater in DevOps especially at automation level 3. Our study shows flexibility and interpersonal skills are important for agile teams (Vivian et al. 2015), and also extends this to others SSk such as teamwork and communication, and suggests that SSk are even more important for DevOps. Our findings are largely congruent with that of Wiedemann and Schulz (2017) and complement this with an in-depth analysis of SSk in Agile and DevOps teams. Beyond key capabilities highlighted by (A. Wiedemann 2017) we identified characteristic features of DevOps teams.

Such a new mode of working “separately together” naturally requires a wider set of skills. Beyond the required specialized skills of one of the two respective worlds, team members need to be more flexible, to have a strong team spirit, and communication (interpersonal) skills to express themselves in a way that will be understandable by the other party and to understand and accept what they are saying when this is correct.

## 5.3 Towards a Maturity Model towards DevOps and Leap-Frogging Implementation Strategies

Our study suggests that as flow automation increases between Dev and Ops, collaboration scope and soft skills also increase, thus allowing knowledge sharing to develop. This finding suggests that there may be different stages of maturity in the transition to DevOps, in which we could consider automation and sharing as two fundamental pillars. Software process maturity frameworks seem to indicate a progression through different steps. Maturity models tend to describe a progression in terms of awareness, richness of activities, knowledge maturation and collective competencies, all resulting in higher performance. The fundamental idea of DevOps is that operational constraints will be integrated by the development team from the outset, and further, for all iterations, code is tested immediately, both

independently and within the ecosystems in which it is embedded. Therefore, in terms of values, transitioning from agile to DevOps means 1) performing testing procedures as far upstream as possible in the development process, 2) *automating* the tests themselves, 3) building common *measurement* of progress and performance, so that ultimately all DevOps team members follow the same dashboard of performance measures, and 4) therefore progressively building a common *culture* based on *shared* mental models, practices and collective action. Our model is a simple one, with only three levels. Concretely, it relies on a progression in terms of level of automation, what is automated, and examines what is shared so that a common ground between Dev and Ops be achieved. Beyond automation and sharing, developing and justifying a maturity model will require considering the other pillars of DevOps (i.e. culture and measurement) and test it in a large population of organizations.

One important practical issue is whether leapfrogging from agility to full DevOps or what could then be considered automation level 1 to automation level 3 can be achieved, or if teams practicing a non-agile, plan driven approach, or just beginning, can start in DevOps mode (Aut.3). One of the teams we observed which is currently the most advanced in innovative agile

practices, started in DevOps mode from its inception in 2010. Research also indicates that companies operating without legacy system can very quickly move all their application portfolio towards continuous delivery (Chen 2015, 2017). Against this backdrop we believe our paper offers a very simple way to imagine a path towards DevOps, as Holmström Olsson et al. (2012) description of barriers on a typical evolution path of transition from waterfall to continuous deployment. However, it does not mean that a direct move to DevOps (Aut.3) is not possible. Depending on the socio-economic activity of the organization, a fast and straight implementation strategy may be risky and prove not sustainable.

## 5.4 Smartness and DevOps Method

The smartness of a socio-technical system refers to the smartness of a set of different dimensions (Alter 2018) including smartness dimensions related to information processing (e.g. capture information, transmit information, display information) and smartness dimensions related to knowledge acquisition (e.g. classifying, compiling, testing and evaluating) (Alter 2018).

**Table 7** IS Function becoming smarter through DevOps

| Smartness Criteria (Alter 2018)                                                                   | DevOps Smartness                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purposefully designed entity                                                                      | DevOps is purposefully created to bridge the gap between Dev and Ops                              | <ul style="list-style-type: none"> <li>• Adaptability, Continuous Delivery (CDY) and Sharing Information evidenced through Skills and Collaborations supporting Smartness</li> <li>• The higher the automation level, the higher the collaborations between Dev and Ops, the better the goal of breaking the silos.</li> <li>• Better collaboration supported by COM and IPS skills slightly higher when automation level increases</li> <li>• MNG and NEG Skills decrease when automation level increases. Teams members are more aligned hence less needs of MNG and NEG skills, improving sharing and CDY.</li> </ul>                               |
| Performs and controls functions                                                                   | DevOps seeks to perform and control two traditionally separate functions as one.                  | <ul style="list-style-type: none"> <li>• Extended collaborations due to the use of tools help CD and sharing, e.g. Automated Batch Processing allows Dev and Ops common way to perform and control</li> <li>• FLX skills are higher when automation level is higher, enabling constant adaptability to perform and control</li> <li>• TWK skills are moderately larger when automation level is higher; team members need to be more aligned together while automation level increases</li> <li>• RES skills decrease when automation level is higher probably due to a transfer from individual responsibilities skills to collective ones</li> </ul> |
| Perceptible results                                                                               | Results are directly perceptible                                                                  | <ul style="list-style-type: none"> <li>• Results evaluated by cross-functional teams with common metrics, common boards to DevOps teams (sharing)</li> <li>• Enables Continuous Delivery and Continuous Deployment with immediate and visible result</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                        |
| Use of automated capabilities for processing, interpreting information, learning from information | Automation at the heart of DevOps.                                                                | <ul style="list-style-type: none"> <li>• Extended sharing due to cross-functional automation --Enables extended possibilities of Knowledge acquisition, learning (FLX), and with tools i.e. Vamp offers a functionality to automate experiments, hence possibilities of adaptability</li> <li>• Allows extended possibilities of processing and interpreting information with automation, i.e. Jira, some features of Amazon Web Services (AWS)</li> </ul>                                                                                                                                                                                             |
| Context dependency                                                                                | Capture and use of information is directly put into context at high level of automation in DevOps | <ul style="list-style-type: none"> <li>• Context-related knowledge is a basis of DevOps system</li> <li>• DevOps enables permanent adaptability to changing environment and context (cf. FLX)</li> <li>• DevOps enlarges team members vision to a wider picture of their environment, hence modifies context dependency, i.e. multiplication of API's</li> </ul>                                                                                                                                                                                                                                                                                       |

In the context of IT companies, the transition to DevOps facilitates the IS function becoming smart for three main reasons (Table 7): adaptability, continuous delivery and sharing information. Firstly, adaptability is considered an indicator of smartness. Agility involves being able to respond quickly and effectively to the current internal and external demands, and to be proactive in developing and retaining markets in the face of extensive competitive forces (Oliveira Martins 2017). Thus, the adaptability of people from Dev and Ops teams is a requirement to efficiently work in a cross functional environment and to consider and understand the needs of different co-workers. Extending collaboration across functions within IS is a sign of adaptation where there is an evident need. A better collaboration between Dev and Ops can be considered as the basis of a successful DevOps. Thus, smartness dimensions related to information processing could refer to the observation of an efficient communication and coordination in teams between Dev and Ops as a unique team. In addition, this efficient communication is supported by the good and smart use of flexible communications tools such as DevOps ceremonies (e.g. daily standup, sprint review).

Second, the fact that delivery can be continuous at the DevOps stage is also a sign of smartness and a sign of adaptation to a more demanding and performant environment. In that sense, smarter is better.

Third, in line with the two previous reasons, DevOps is the agile approach whereby better sharing of information and developing a common culture can happen across different roles and jobs throughout the IS function as a whole which overcomes the traditional distinction between Dev and Ops. With such outcomes people understand each other better and extend their horizons and possibly their learning capabilities. Accordingly, the dimension related to knowledge acquisition shows a high level of smartness through the “testing and evaluation” done in DevOps team, during, for example, the phase of demonstration between Dev and Ops.

## 6 Conclusion

In this study, we discussed the extent to which progression towards DevOps reflects greater smartness. We identified three different stages underpinning the transition from Agile to DevOps, i.e. Agile (Aut.1) Continuous Integration (Aut.2), Continuous Delivery (Aut.3). We examined the importance of skills and collaboration patterns in the progression towards DevOps.

We identified a fundamental disruption in the soft skills that software teams are expected to possess, and in the patterns of collaboration within teams. We revealed that the collaborative scope within the main collaborators clearly evolved and is more balanced between Dev and Ops at automation level 3. We explained this result by richer collaborations reflecting better understandings among members. We also described three stages which

are characterized by additional collaboration in line with the increasing level of automation, and an evolution of perceived skills for each stage. We showed the importance of responsibilities and communication skills at automation level 1, which is expected because Agile Methodologies foster and support these skills. Surprisingly, flexibility and teamwork skills were less evident in the interviews. Then, at the next level, we found an extension of these collaborations with the exception of skills related to responsibilities. These were decreasing at automation level 3 also. We used Alter’s (2018) smartness criteria to investigate DevOps smartness. We provided evidence that DevOps fostered automation, developed flexibility skills and produced perceptible results such as faster and better deliveries. Consequently, DevOps may lead to greater smartness for the Information System function according to Alter’s smartness conceptualization (Alter 2018).

Although previous studies developed maturity models in Software Engineering (Curley et al. 2016; Luftman and Kempaiah 2007; Paulk et al. 1993), no such model has been developed to analyze the transition from Agile to DevOps. A commercial organization, Atlassian-GitLab, has proposed a simplified DevOps Maturity Model which is more technical in focus. Also, it does not provide any guidance on how to manage the transition to DevOps or its direct full adoption. Few researchers have analyzed DevOps practices (Lwakatare et al. 2016). We are not aware of any research outlining the conceptual or theoretical underpinnings of this transition to DevOps, nor of any research focused on the relationships between smartness and DevOps.

In general, software process maturity frameworks indicate a progression through different steps. The stages of maturity show a progression in terms of level of automation, what is automated, what is shared and measured. Then, our paper presented practical issues with different options, whether leapfrogging from automation level 1 to automation level 3 can be achieved, or if teams practicing a waterfall plan-driven approach can start in automation level 3 from the outset. The three identified stages provide a very simple way to imagine a path towards DevOps, but as we highlighted, it means that a straight move to DevOps is possible. Overall, the IS function can be deemed smarter through DevOps.

One limitation of our research is related to the choice of our methodology: an exploratory embedded case-study based on a stratified and rich sample of teams in a single large company. Therefore, it is possible that our findings would vary if a new analysis is realized in another company. These findings may be limited to this firm and to this specific large European firm’s context. Another limitation of our study is that the extent of more diverse collaboration was not fully demonstrated with our methodology. A method focusing on observations and who collaborates with whom would be valuable to refine our findings. Finally, it would be interesting to go deeper in the analysis to identify elements that could limit DevOps smartness which are not necessarily the opposite of the factors generating the smartness.

## Appendix 1

**Table 8** Index

| Acronym | Full term                           |
|---------|-------------------------------------|
| AR      | Architect                           |
| ARC     | Architecture                        |
| AWS     | Amazon Web Services                 |
| CD      | Continuous Deployment               |
| CDY     | Continuous Delivery                 |
| CI      | Continuous Integration              |
| COM     | Communication                       |
| CPL     | Coding and Programming Languages    |
| CSY     | Courtesy                            |
| Dev     | Development Function                |
| DPM     | Department & Project Manager        |
| FLX     | Flexibility Skills                  |
| HSk     | Hard Skills                         |
| IDEs    | Integrated Development Environments |
| IPS     | Interpersonal Skills                |
| ITG     | Integrity                           |
| MNG     | Management                          |
| NEG     | Negotiation                         |
| Ops     | Operations Function                 |
| PAT     | Positive Attitude                   |
| PE      | Production Engineers                |
| PO      | Product Owner                       |
| PRO     | Professionalism                     |
| QSY     | Quality, Security                   |
| RES     | Responsibility                      |
| RM      | Release Manager                     |
| RMT     | Release Management                  |
| SSk     | Soft Skills                         |
| SWE     | Software Engineering                |
| SYS     | Systems                             |
| TST     | Testing                             |
| TWK     | Teamwork                            |
| WKE     | Work Ethic                          |



## Appendix 2

**Table 9** Coding example

| Example of transcripts on skills (excerpts)                                                                                                                                                                                                                        | Codes                                         | Emerging themes                          | Final themes                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------|-------------------------------------|
| You do not have to be afraid to <b>seek</b> , to <b>look for</b> and go on platforms, on the Internet. Because there are a lot of <b>problems</b> (...) which are not <b>documented</b> and for which we have to do dedicated research                             | Investigate<br>Problem<br>Documentation       | Resourceful<br><br>Problem solving       | Responsibility<br>Analytical Skill  |
| You must be <b>humble</b> or agree to say that you do <b>not know</b> , and that your neighbor, he knows better than you and that he will show you. Not too much trying not to <b>impose</b> your vision either.                                                   | Humility<br>Don't Know<br>Impose vision       | Lifelong learner<br>Tolerant, respectful | Flexibility<br>Interpersonal skills |
| <b>Adaptation!</b> We will have to adapt fairly <b>quickly</b> with technological <b>changes</b> .                                                                                                                                                                 | Adaptation<br>Speed<br>Technological changes  | Adaptability                             | Flexibility                         |
| I share. It is also necessary to be ready to <b>integrate</b> the team even if one is manager of some of them. It is also necessary to know how to <b>step aside</b> .                                                                                             | Sharing, Team<br>Integration<br>Less visible  | Cooperative<br>Get along with others     | Teamwork                            |
| I think we should not be afraid of <b>change</b> .                                                                                                                                                                                                                 | No fear<br>Change                             | Accepts new things                       | Flexibility                         |
| You really need to have the <b>vision</b> of <b>belonging</b> to the same <b>team</b> , and not to say: “Me, I represent the <b>Operations</b> side” (...) to feel “ <b>powerful</b> ”, because we have the decision in fine to put the application in production. | Vision, Team Belonging<br>Operations<br>Power | Team spirit                              | Teamwork                            |

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

## References

- Alter, S. (2018). Making sense of smart living, working, and organizing enhanced by supposedly smart objects and systems. In *Proceedings of the IFIP WG8.6 Working Conference: Smart Working, Living and Organizing* (pp. 1–14). Presented at the IFIP WG8.6 working conference: Smart working, living and organizing, Portsmouth. UK: Springer.
- Ambler, S. W., & Lines, M. (2012). *Disciplined agile delivery: A practitioner's guide to agile software delivery in the enterprise*. Indianapolis, IN, USA: IBM Press.
- Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2016). Microservices architecture enables devops: Migration to a cloud-native architecture. *IEEE Software*, 33(3), 42–52.
- Bang, S. K., Chung, S., Choh, Y., & Dupuis, M. (2013). A grounded theory analysis of modern web applications: knowledge, skills, and abilities for DevOps. In *Proceedings of the RIIT, 2nd Annual Conference on Research in Information Technology* (pp. 61–62). Orlando, FL, USA: ACM.
- Betz, C., Olagunju, A. O., & Paulson, P. (2016). The impacts of digital transformation, agile, and DevOps on future IT curricula. In *Proceedings of the 17th Annual Conference on Information Technology Education* (p. 106). Boston, MA, USA: ACM.
- Bjarnason, E., Wnuk, K., & Regnell, B. (2011). Requirements are slipping through the gaps—A case study on causes & effects of communication gaps in large-scale software development. In *2011 19th IEEE International Requirements Engineering Conference (RE)* (pp. 37–46). IEEE.
- Boyatzis, R. E., & Kolb, D. A. (1995). From learning styles to learning skills: The executive skills profile. *Journal of Managerial Psychology*, 10(5), 3–17.
- Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101.
- Brunnert, A., van Hoorn, A., Willnecker, F., Danciu, A., Hasselbring, W., Heger, C., et al. (2015). Performance-oriented DevOps: A research agenda. *arXiv preprint arXiv: 1508.04752*.
- Bruns, H. C. (2013). Working alone together: Coordination in collaboration across domains of expertise. *Academy of Management Journal*, 56(1), 62–83.
- Chen, L. (2015). Continuous delivery: Huge benefits, but challenges too. *IEEE Software*, 32(2), 50–54.
- Chen, L. (2017). Continuous delivery: Overcoming adoption challenges. *Journal of Systems and Software*, 128, 72–86.
- Conboy, K., & Coyle, S. (2009). People over process: The implications of agile for IS skills and human resource management. *IEEE Software*, 28, 48–57.
- Conboy, K., & Fitzgerald, B. (2004). Toward a conceptual framework of agile methods: A study of agility in different disciplines. In *Proceedings of the 2004 ACM workshop on Interdisciplinary software engineering research* (pp. 37–44). Newport Beach, CA, USA: ACM.
- Conboy, K., Coyle, S., Wang, X., & Pikkarainen, M. (2011). People over process: Key people challenges in agile development. *IEEE Software*, 28, 48–57.
- Coyle, S., Conboy, K., & Acton, T. (2015). An exploration of the relationship between contribution behaviours and the decision making process in agile teams. In *Proceedings of the 36th ICIS international*

- conference on information systems (pp. 1–15). USA: Fort Worth, TX.
- Curley, M., Kenneally, J., & Carcary, M. (2016). IT capability maturity framework™(IT-CMF™). In *Zalibommel*. Netherlands: Van Haren.
- Debois, P. (2009). DevOpsDaysGhent. Presented at the DevOpsDays, Ghent, Belgium. <http://www.devopsdays.org/events/2009-ghent/>
- Debois, P. (2011). Devops: A software revolution in the making. *Journal of Information Technology Management*, 24(8), 3–39.
- Dingsøyr, T., Faegri, T. E., & Itkonen, J. (2014). What is large in large-scale? A taxonomy of scale for agile software development. In *International Conference on Product-Focused Software Process Improvement* (pp. 273–276). Springer.
- Elshamy, A., & Elssamadisy, A. (2007). Applying agile to large projects: new agile software development practices for large projects. In *International Conference on Extreme Programming and Agile Processes in Software Engineering* (pp. 46–53). Springer.
- Erich, F. M. A., Amrit, C., & Daneva, M. (2017). A qualitative study of DevOps usage in practice. *Journal of Software: Evolution and Process*, 29(6), e1885, 1–47.
- Fitzgerald, B., & Stol, K.-J. (2017). Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software*, 123, 176–189.
- Forrester Research. (2015). Infrastructure as code: Fueling the fire for faster application delivery. Forrester Consulting.
- Gallagher, K. P., Kaiser, K. M., Simon, J. C., Beath, C. M., & Goles, T. (2010). The requisite variety of skills for IT professionals. *Communications of the ACM*, 53(6), 144–148.
- Gallivan, M. J., Truex, D. P., III, & Kvasny, L. (2004). Changing patterns in IT skill sets 1988–2003: A content analysis of classified advertising. *ACM SIGMIS Database: the DATABASE for Advances in Information Systems*, 35(3), 64–87.
- Garvin, D. A. (2013). How Google sold its engineers on management. *Harvard Business Review*, 91(12), 75–82.
- Goles, T., Hawk, S., & Kaiser, K. M. (2008). Information technology workforce skills: The software and IT services provider perspective. *Information Systems Frontiers*, 10(2), 179–194.
- Gren, L., Knauss, A., & Stettina, C. J. (2018). Non-technical individual skills are weakly connected to the maturity of agile practices. *Information and Software Technology*, 99, 11–20.
- Hannay, J. E., & Benestad, H. C. (2010). Perceived productivity threats in large agile development projects. In *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement* (p. 15). ACM.
- Hemon, A., Monnier-Senicourt, L., & Rowe, F. (2018). Job satisfaction factors and risks perception: an embedded case study of DevOps and agile teams. In *Proceedings of the 39th International Conference on Information Systems (ICIS)*, San Francisco, CA, USA.
- Highsmith, J., & Cockburn, A. (2001). Agile software development: The business of innovation. *Computer*, 34(9), 120–127.
- Holmström Olsson, H., Alahyari, H., & Bosch, J. (2012). Climbing the "Stairway to Heaven"—A Multiple-Case Study Exploring Barriers in the Transition from Agile Development towards Continuous Deployment of Software. In *2012 38th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 392–399). IEEE.
- Humble, J., & Molesky, J. (2011). Why enterprises must adopt devops to enable continuous delivery. *Cutter IT Journal*, 24(8), 6.
- Hussain, W., Clear, T., & MacDonell, S. (2017). Emerging trends for global DevOps: a New Zealand perspective. In *Proceedings of the 12th ICGSE International Conference on Global Software Engineering* (pp. 21–30). Buenos Aires, Argentina: IEEE Press.
- Hüttermann, M. (2012). *DevOps for Developers* (1st ed.). New York, NY, USA: Apress.
- Kalliamvakou, E., Bird, C., Zimmermann, T., Begel, A., DeLine, R., & German, D. M. (2017). What makes a great manager of software engineers? *IEEE Transactions on Software Engineering*, 1–20.
- Kerzazi, N., & Adams, B. (2016). Who needs release and devops engineers, and why? In *Proceedings of the International Workshop on Continuous Software Evolution and Delivery* (pp. 77–83). Austin, TX, USA: ACM.
- Kim, G., Debois, P., Willis, J., & Humble, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland. In OR, USA: IT Revolution.
- Lincoln, Y. S., & Guba, E. G. (1985). *Naturalistic inquiry*. Beverly Hills: Sage Publications.
- Luftman, J., & Kempaiah, R. (2007). An update on business-IT alignment: "a line" has been drawn. *MIS Quarterly Executive*, 6(3), 165–177.
- Lwakatare, L. E., Kuvaja, P., & Oivo, M. (2015). Dimensions of devops. In *International Conference on Agile Software Development* (pp. 212–217). Helsinki, Finland: Springer.
- Lwakatare, L. E., Kuvaja, P., & Oivo, M. (2016). An exploratory study of DevOps extending the dimensions of DevOps with practices. In *Proceedings of the 11th international conference on software engineering advances (ICSEA)* (pp. 91–99). Rome, Italy: IARIA.
- Maram, M., Prabhakaran, P., Murthy, S., & Domala, N. (2009). Sixteen roles performed by software engineers in first one year. In *22nd Conference on Software Engineering Education and Training* (pp. 212–215). IEEE.
- Nerkar, A., & Paruchuri, S. (2005). Evolution of R&D capabilities: The role of knowledge networks within a firm. *Management Science*, 51(5), 771–785.
- Nerur, S., Mahapatra, R., & Mangalaraj, G. (2005). Challenges of migrating to agile methodologies. *Communications of the ACM*, 48(5), 72–78.
- Oliveira Martins, S. R. (2017). High Radicality of product innovation and high flexibility and high agility of system of manufacturing: Towards the smart factories. *Procedia Manufacturing*, 11, 1324–1334.
- Oomen, S., De Waal, B., Albertin, A., & Ravesteyn, P. (2017). How can scrum be successful? Competences of the scrum product owner. In *Proceedings of the 25th ECIS European Conference on Information Systems* (pp. 131–142). Guimarães, Portugal.
- Paulk, M. C., Curtis, B., Chrissis, M. B., & Weber, C. V. (1993). The capability maturity model for software. *Software engineering project management*, 10, 1–26.
- Petersen, K., & Wohlin, C. (2009). A comparison of issues and advantages in agile and incremental development between state of the art and an industrial case. *Journal of Systems and Software*, 82(9), 1479–1490.
- Robles, M. M. (2012). Executive perceptions of the top 10 soft skills needed in today's workplace. *Business Communication Quarterly*, 75(4), 453–465.
- Rolland, K. H., Fitzgerald, B., Dingsøyr, T., & Stol, K.-J. (2016). Problematising agile in the large: Alternative assumptions for large-scale agile development. In *Proceedings of the 37th ICIS International Conference on Information Systems*, Ireland, Dublin (pp. 1–21).
- Runeson, P., & Höst, M. (2008). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131.
- Schwaber, K., & Sutherland, J. (2011). The scrum guide. *Scrum Alliance*, 21.
- Stahl, D., & Bosch, J. (2014). Modeling continuous integration practice differences in industry software development. *Journal of Systems and Software*, 87, 48–59.
- Strode, D. E. (2016). A dependency taxonomy for agile software development projects. *Information Systems Frontiers*, 18(1), 23–46.
- Strode, D. E., Huff, S. L., & Tretiakov, A. (2009). The impact of organizational culture on agile method use. In *42nd Hawaii International Conference on System Sciences (HICSS)* (pp. 1–9). IEEE.
- Tan, C.-H., & Teo, H.-H. (2007). Training future software developers to acquire agile development skills. *Communications of the ACM*, 50(12), 97–98.
- Todd, P. A., McKeen, J. D., & Gallupe, R. B. (1995). The evolution of IS job skills: A content analysis of IS job advertisements from 1970 to 1990. *MIS Quarterly*, 19(1), 1–27.

- Trauth, E. M., Farwell, D. W., & Lee, D. (1993). The IS expectation gap: Industry expectations versus academic preparation. *MIS Quarterly*, 17, 293–307.
- Turley, R. T., & Bieman, J. M. (1994). Identifying essential competencies of software engineers. In *ACM Conference on computer science*, 271–278.
- Version One. (2018). 12th annual state of agile report. (April 2018) (p. 15). Alpharetta, GA, USA. <https://explore.versionone.com/state-of-agile/versionone-12th-annual-state-of-agile-report>
- Vivian, R., Tarmazdi, H., Falkner, K., Falkner, N., & Szabo, C. (2015). The development of a dashboard tool for visualising online team-work discussions. In *IEEE/ACM 37th IEEE International Conference on Software Engineering* (Vol. 2, pp. 380–388). IEEE.
- Wettinger, J., Breitenbücher, U., & Leymann, F. (2014). Standards-based DevOps automation and integration using TOSCA. In *Proceedings of the 2014 IEEE/ACM* (pp. 59–68). Presented at the 7th International Conference on Utility and Cloud Computing, London, UK: IEEE Computer Society.
- Wiedemann, A. (2017). A New Form of Collaboration in IT Teams-Exploring the DevOps Phenomenon. In *Proceedings of the 21st PACIS Pacific Asia Conference on Information Systems* (Vol. 82, pp. 1–12). Langkawi.
- Wiedemann, A. M., & Schulz, T. (2017). Key Capabilities of DevOps Teams and their Influence on Software Process Innovation: A Resource-Based View. In *Proceedings of 23rd ACIS Americas Conference on Information Systems* (pp. 1–10). Boston.
- Wong, S., von Hellens, L., & Orr, J. (2006). Non-technical skills and personal attributes: The soft skills matter Most. In *Proceedings of the 6th Australasian Women in Computing Workshop* (pp. 27–33). Brisbane, Australia.
- Yin, R. K. (1994). *Case study research: design and methods* (2nd edn.). Thousand oaks, CA, USA: Sage Publications.

**Aymeric Hemon** is Assistant Professor in Information Systems at ESSCA School of Management and associate researcher at the LEMNA Laboratory, University of Nantes. He holds a Ph.D. in MIS from the University of Nantes. His research focus on IS-enabled organizational

transformation, particularly on agile methods, DevOps and automation. Before taking on an academic position, he worked principally in the fields of corporate reorganization for about 10 years.

**Barbara Lyonnet** is Associate Professor in Management Sciences at University of Nantes and researcher at LEMNA Laboratory, France. She is Doctor of Engineering Science. Her research interests are in information system, quality management and supply chain management. Her work has been published in leading academic journals such as *Production Planning and Control*, *International Journal of Business Excellence*, *International Journal of Computer Theory and Engineering*. She has also written several significant books in the field of logistics and lean management. She has participated and presented her research at various International conferences like International Conference on Advanced Manufacturing and Industrial Application. She is or has been member of the Scientific Committee of several international conferences including the Business Sustainability Conference, International Conference on Computer Modeling and Simulation and International Conference on Traffic and Logistic Engineering.

**Frantz Rowe** is Professor in Information Systems at University of Nantes and researcher at LEMNA Laboratory, and at KTO (SKEMA Business School), France. He has been the CIO of his University and is now Editor Emeritus of the European Journal of Information Systems (EJIS). He recently published two papers on causality in EJIS and MIS Quarterly and has been leading the Research Program on DevOps at LEMNA. His research interests also include the philosophy of information systems, IS-enabled organizational transformation, and methods related to literature reviews. He is a Fellow of the AIS.

**Brian Fitzgerald** is Director of Lero – the Irish Software Research Centre. He also holds an endowed professorship, the Krehbiel Chair in Innovation in Business & Technology, at the University of Limerick, Ireland. His publications include 16 books, and over 150 peer-reviewed articles in the leading international journals and conferences in both the Information Systems and Software Engineering fields. Prior to taking up an academic position, he worked in the software industry for about 12 years.